



OUTLIERS

When You Are Not There

Layer 5 of 5 · It runs without you,
and you find out whether it did.

EST. MMXXVI · OUTLIERS ACCELERATOR

Getting used to it being on

This is the layer where the machine stops being something you operate and becomes something you manage. The timetable is the easy half. Knowing whether anything happened while you were not watching is the half nobody expects.

WHAT THE LAST LAYER LEFT YOU WITH

Two platforms, both working, and you are still the one typing every command. Nothing happens unless you are sitting there, which means it happens on the days you remember and not on the days you are busy.

WHAT THIS LAYER BUILDS

A timetable that runs any command on the days and inside the hours you set, and keeps running after you close the window. Then the part that matters more: a way to tell the difference between a job that found nothing and a job that broke.

FROM OUTSIDE, THESE TWO LOOK IDENTICAL

```
RAN, AND FOUND NOBODY  
finished cleanly  
reported success
```

```
BROKE ON THE FIRST PAGE  
finished cleanly  
reported success
```

WHAT TELLS THEM APART

```
read 0 • written 0 • already there 0 • could not place 0  
a job that read nothing and a job that read 200 and knew them all are not the same result
```

A month of runs that found nothing looks, from every log, exactly like a month of working perfectly.

BOTH FINISHED. ONLY ONE WORKED.

How it works

Start with one job. The supervision is the reason, not the extra.

START WITH ONE JOB, ONCE A DAY

Not a schedule. One entry, so the first time the machine acts on its own it is one small readable event you can go and look at afterwards. Add the second when the first has been running long enough to be boring.

IT NEVER RUNS AT THE SAME MINUTE TWO DAYS RUNNING

Each job's time shifts slightly, worked out from the job and the date rather than picked at random. Being regular is a signature on its own, and a timetable is the easiest way to become regular by accident. Worked out rather than random matters: if the machine restarts it arrives at the same times again, so nothing drifts and nothing fires twice.

THEN THE PART NOBODY EXPECTS

An automatic job fails quietly by default. It carries on running, reporting success, and producing nothing, because "found nothing to do" is a perfectly successful run. A month of those looks identical from every log to a month of working.

And the supervision can lie in both directions. Once it reported a job as hung for two days when the job had started that second, because it was reading a note the previous run had left behind. Later it reported a job as running when the job had been dead for two days, because the number identifying it had been handed on to something else entirely, and it then refused to start the real one, because it believed one was already going. The first is annoying. The second hides the failure and blocks the cure.

WHAT ACTUALLY TELLS YOU

Not whether it ran. What it brought back. How many it read, how many were written down, how many you already had, and how many it could not place. Your collectors already print all four, so there is nothing here for you to build.

Build it

One command. It asks you a few questions so what you get is yours, then does the work.

```
git clone https://github.com/OUTLIERS-ai/outliers-gather-05-timetable
cd outliers-gather-05-timetable
python install.py
```

This runs any command, not only the ones in this set. Nothing reaches the outside world during installation, and it schedules nothing until you add an entry.

WHAT IT ASKS YOU, AND WHY

QUESTION	WHAT IT CHANGES
What is the first command you want it to run?	One to begin with. Anything you would type yourself, not only the jobs in this set.
At what time?	A starting point rather than an exact minute. The minute is shifted on purpose so it is never the same two days running.
On which days?	The days it may fire at all.
What would you call it?	A name you will recognise in a list months from now, when you have forgotten writing it.
Should it start every time you log in?	Answer no until you have watched it run once while you were there. It can be turned on later with one command.

WHAT YOU END UP WITH

- A timetable you can read, holding what runs and when.
- Times that vary slightly but never drift.
- It keeps running after the window is closed. **install-startup** is a separate command, for when you want it back after a restart as well.
- Each job may carry its own time limit, set with **--minutes** when you add it. One limit shared by everything always cuts the slowest job short.
- No window ever opens over whatever you are doing.

Now use it

Add one job. Watch what it says before it ever runs.

In the **_engine** folder inside your CRM:

```
python timetable.py add "python gather.py undo" --at 09:30 --days mon,wed,fri
python timetable.py run --dry-run
```

The second prints what would fire today and at exactly what time, and fires none of it. The minute will not be 09:30, and it will be the same minute if you run it again, and that is the variation working rather than a fault.

When you are ready, start it properly and leave it. Then come back tomorrow and look at what it brought in rather than at whether it ran.

IF YOU ALREADY HAD SOMETHING LIKE THIS

A test rather than an instruction. Whatever you already run on a schedule: how would you find out today if it had quietly stopped a fortnight ago? If the answer is that you would notice the results drying up, you would not.

WHAT THIS LAYER LEAVES UNSOLVED

Everything across these five layers brings information *in*. Nothing has gone *out*: nothing has typed at anybody, and nothing has sent a message. That has been deliberate from the first page.

WHAT THIS WAS PREPARING FOR

Content, conversations and messages: the machine talking to the world rather than reading it. It comes after this because an account restriction lifts in a day or two if you stop, and a message sitting in somebody's inbox does not. You have spent five layers finding out whether you can trust this running unwatched on work where being wrong costs nothing.

WORDS USED IN THIS GUIDE

Timetable	The list of what runs and when.
Unattended	Running without anybody watching. Everything above assumes you are; this does not.